

CUADRADA 1 FFT

```
%  
  
% GRAFICACIÓN DE FFT CON MARCADORES  
  
%  
  
clear; clc; close all;  
  
%% 1. CONFIGURACIÓN Y LECTURA DEL ARCHIVO CSV  
  
filename = 'cuad1fft.csv';  
  
% Verificar la existencia del archivo  
if ~exist(filename, 'file')  
    error('El archivo %s no se encuentra en el directorio actual.', filename);  
end  
  
% Abrir y procesar linea por linea para ignorar metadatos del osciloscopio  
  
fid = fopen(filename, 'r');  
  
X = []; % Frecuencia (Hz)  
Y = []; % Magnitud (dB)  
  
while ~feof(fid)  
    linea = fgetl(fid);  
    partes = strsplit(linea, ',');  
    nums = str2double(partes);  
    validos = nums(~isnan(nums));  
  
    % Extraer los dos últimos valores numéricos presentes en cada fila  
    if length(validos) >= 2  
        X(end+1, 1) = validos(end-1);  
        Y(end+1, 1) = validos(end);  
    end  
end  
  
end
```

```

fclose(fid);

%% 2. DETECCIÓN DE PICOS / MARCADORES

% Ajustar 'MinPeakProminence' para controlar la cantidad de picos marcados
min_prominence = 15; % Prominencia minima en dB
min_height = -30; % Altura minima en d
[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);
picos_X = X(locs);

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA
fig = figure('Name', 'Análisis Espectral FFT - cuad1fft.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1000, 500]);

hold on;

% Graficar trazo de la señal FFT
plot(X, Y, 'Color', [0 0.4470 0.7410], 'LineWidth', 1.2, ...
    'DisplayName', 'Espectro FFT (dB)');

% Graficar marcadores en los picos detectados
plot(picos_X, picos_Y, 'x', ...
    'Color', [0.8500 0.3250 0.0980], ...
    'LineWidth', 1.5, ...
    'MarkerSize', 8, ...
    'DisplayName', 'Marcadores (Picos)');

%% 4. ETIQUETADO DE LOS MARCADORES EN LA GRÁFICA
for i = 1:length(picos_X)
    etiqueta = sprintf('%0.1f Hz\n%0.1f dB', picos_X(i), picos_Y(i));

```

```

text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...

'FontSize', 7.5, ...

'Color', [0.2 0.2 0.2], ...

'HorizontalAlignment', 'center', ...

'VerticalAlignment', 'bottom');

end

%% 5. FORMATO FINAL Y ESTÉTICA DE LA GRÁFICA

grid on;

grid minor;

box on;

xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');

ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');

title('Espectro de Frecuencia (FFT) y Marcadores de Armónicos', ...

'FontSize', 12, 'FontWeight', 'bold');

legend('Location', 'northeast');

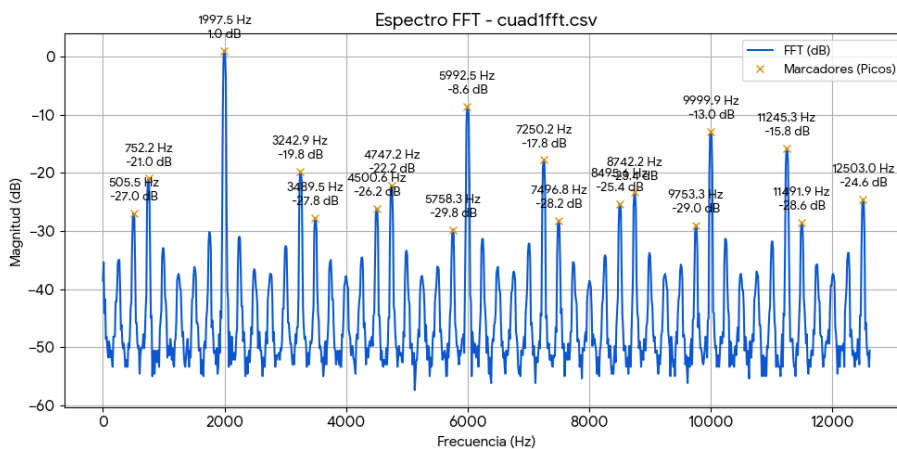
% Ajuste dinámico de límites del eje

xlim([0, max(X)]);

ylim([min(Y) - 5, max(Y) + 10]);

hold off;

```



CUADRADA 1 NORMAL

```
%  
  
% SEÑAL TEMPORAL CON MARCADORES (cuad1normal.csv)  
  
%  
  
clear; clc; close all;  
  
%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV  
  
filename = 'cuad1normal.csv';  
if ~exist(filename, 'file')  
    error('El archivo %s no se encuentra en el directorio actual.', filename);  
end  
  
fid = fopen(filename, 'r');  
  
t_raw = []; % Tiempo (s)  
v_raw = []; % Voltaje (V)  
  
while ~feof(fid)  
    linea = fgetl(fid);  
    partes = strsplit(linea, ',');  
    nums = str2double(partes);  
    validos = nums(~isnan(nums));  
  
    % Extrae los últimos dos valores numéricos de cada fila (Tiempo y Voltaje)  
    if length(validos) >= 2  
        t_raw(end+1, 1) = validos(end-1);  
        v_raw(end+1, 1) = validos(end);  
    end  
end  
  
fclose(fid);  
  
% Convertir tiempo a milisegundos (ms) para facilitar lectura
```

```

t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE PARÁMETROS Y UBICACIÓN DE MARCADORES

[v_max, idx_max] = max(v_raw);
[v_min, idx_min] = min(v_raw);
v_pp = v_max - v_min;

% Detección de cruces por cero (flancos de subida)
cruces_idx = find(diff(sign(v_raw)) > 0);
t_cruces_ms = t_ms(cruces_idx);

% Cálculo de Período y Frecuencia
if length(t_cruces_ms) > 1
    periodo_ms = mean(diff(t_cruces_ms));
    frecuencia_hz = 1 / (periodo_ms * 1e-3);
else
    periodo_ms = NaN;
    frecuencia_hz = NaN;
end

%% 3. GRAFICACIÓN DE LA SEÑAL Y MARCADORES

figure('Name', 'Señal en el Tiempo - cuad1normal.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1000, 520]);

hold on;

% Trazo principal de la señal
plot(t_ms, v_raw, 'b-', 'LineWidth', 1.3, 'DisplayName', 'Señal Cuadrada (CH1)');

% Marcador de Voltaje Máximo
plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));

```

```

% Marcador de Voltaje Mínimo

plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...
     'DisplayName', sprintf('V_{mín} = %.2f V', v_min));

% Marcadores en flancos de subida (cruces por cero)

plot(t_cruces_ms, zeros(size(t_cruces_ms)), 'k^', 'MarkerFaceColor', 'k', ...
     'MarkerSize', 6, 'DisplayName', 'Flancos de subida');

% Línea de nivel cero

yline(0, '--k', 'Alpha', 0.4, 'HandleVisibility', 'off');

%% 4. ANOTACIONES Y ETIQUETAS

% Etiqueta Vmax

text(t_ms(idx_max), v_max + 0.15, sprintf(' V_{máx} = %.2f V', v_max), ...
     'Color', 'r', 'FontWeight', 'bold', 'FontSize', 9);

% Etiqueta Vmin

text(t_ms(idx_min), v_min - 0.15, sprintf(' V_{mín} = %.2f V', v_min), ...
     'Color', [0 0.5 0], 'FontWeight', 'bold', 'FontSize', 9);

% Cuadro con métricas clave (Vpp, Período, Frecuencia)

info_str = {
    sprintf('V_{pp} = %.2f V', v_pp), ...
    sprintf('Período (T) = %.3f ms', periodo_ms), ...
    sprintf('Frecuencia (f) = %.2f kHz', frecuencia_hz / 1e3)
};

annotation('textbox', [0.15, 0.15, 0.25, 0.15], 'String', info_str, ...
     'FitBoxToText', 'on', 'BackgroundColor', 'w', ...
     'EdgeColor', [0.5 0.5 0.5], 'FontSize', 9);

%% 5. FORMATO Y ESTÉTICA

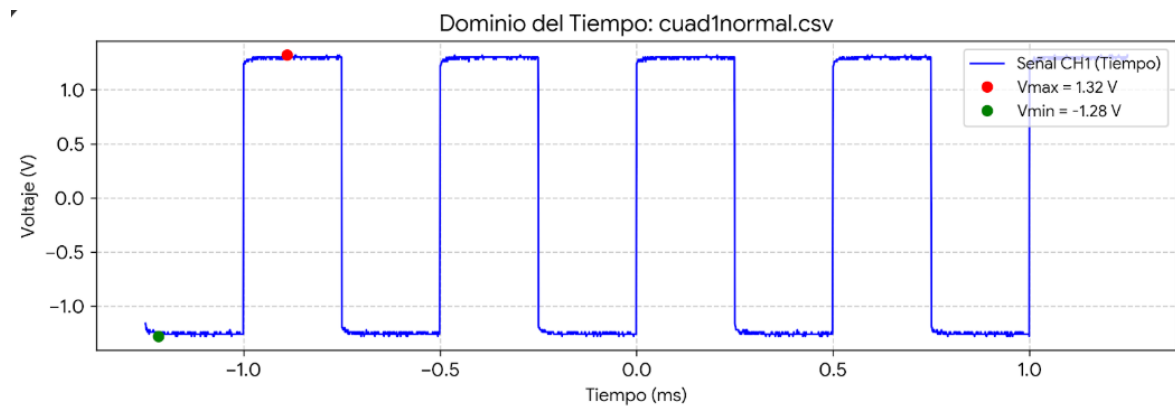
grid on; grid minor; box on;

```

```

xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');
title('Análisis de Señal Cuadrada en el Dominio del Tiempo', ...
      'FontSize', 12, 'FontWeight', 'bold');
legend('Location', 'northeast');
xlim([min(t_ms) max(t_ms)]);
ylim([v_min - 0.4, v_max + 0.4]);
hold off;

```



CUADRADA 2 FFT

```

%
% GRAFICACIÓN DE FFT CON MARCADORES (cuad2fft.csv)
%
clear; clc; close all;

%% 1. LECTURA Y EXTRACCIÓN ROBUSTA DE DATOS

filename = 'cuad2fft.csv';

if ~exist(filename, 'file')
    error('El archivo %s no se encuentra en el directorio actual.', filename);
end

fid = fopen(filename, 'r');

X = []; % Frecuencia (Hz)

```

```

Y = []; % Magnitud (dB)

while ~feof(fid)

    linea = fgetl(fid);

    partes = strsplit(linea, ',');

    nums = str2double(partes);

    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)

    if length(validos) >= 2

        X(end+1, 1) = validos(end-1);

        Y(end+1, 1) = validos(end);

    end

end

fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)

% Se filtran picos relevantes con prominencia mínima de 15 dB

min_prominence = 15;

min_height = -30;

[picos_Y, locs] = findpeaks(Y, ...

    'MinPeakProminence', min_prominence, ...

    'MinPeakHeight', min_height);

picos_X = X(locs);

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA

figure('Name', 'Espectro FFT - cuad2fft.csv', ...

    'Color', 'w', ...

    'Position', [100, 100, 1000, 500]);

hold on;

```



```

% Trazo principal del espectro FFT
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');

% Marcadores sobre los picos armónicos
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...
     'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');

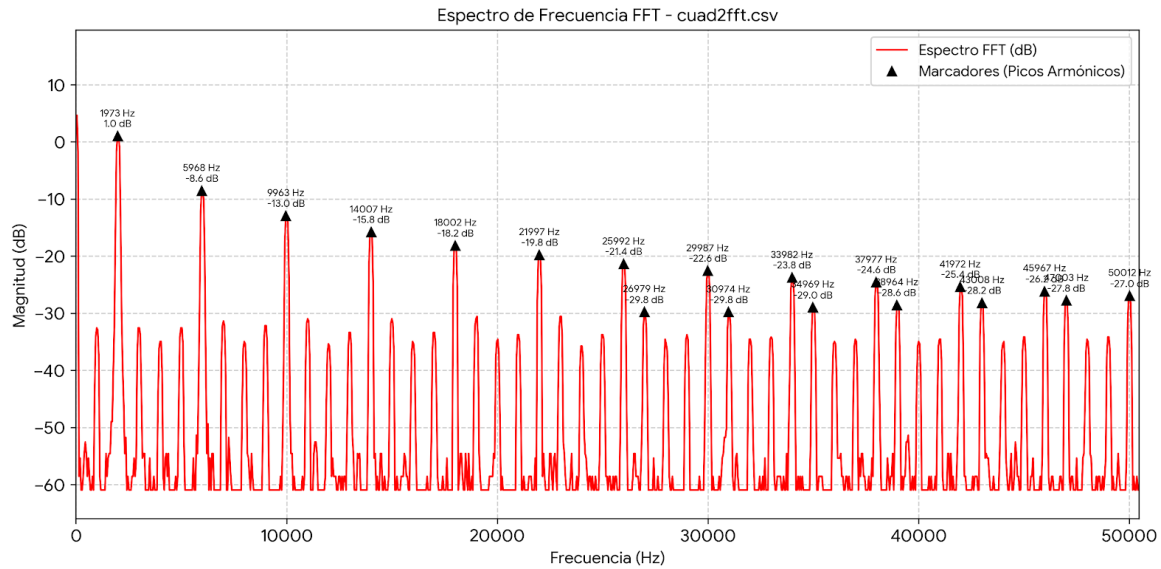
%% 4. ETIQUETADO DE CADA MARCADOR
for i = 1:length(picos_X)
    etiqueta = sprintf('%0.0f Hz\n%0.1f dB', picos_X(i), picos_Y(i));
    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...
         'FontSize', 7, ...
         'Color', 'k', ...
         'HorizontalAlignment', 'center', ...
         'VerticalAlignment', 'bottom');
end

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA
grid on; grid minor; box on;
xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');
title('Espectro de Frecuencia FFT - cuad2fft.csv', ...
      'FontSize', 12, 'FontWeight', 'bold');
legend('Location', 'northeast');

% Ajuste de límites ejes
xlim([0, max(X)]);
ylim([min(Y) - 5, max(Y) + 15]);

hold off;

```



CUADRADA 2 NORMAL

%

% SEÑAL TEMPORAL CON MARCADORES (cuad2normal.csv)

%

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV

filename = 'cuad2normal.csv';

if ~exist(filename, 'file')

error('El archivo %s no se encuentra en el directorio actual.', filename);

end

fid = fopen(filename, 'r');

t_raw = []; % Tiempo (s)

v_raw = []; % Voltaje (V)

while ~feof(fid)

linea = fgetl(fid);

partes = strsplit(linea, ',');

nums = str2double(partes);

```

validos = nums(~isnan(nums));

% Extrae los últimos dos valores numéricos de cada fila
if length(validos) >= 2
    t_raw(end+1, 1) = validos(end-1);
    v_raw(end+1, 1) = validos(end);
end

end

fclose(fid);

% Convertir tiempo a milisegundos (ms)
t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE MÉTRICAS Y UBICACIÓN DE MARCADORES
[v_max, idx_max] = max(v_raw);
[v_min, idx_min] = min(v_raw);
v_pp = v_max - v_min;

% Umbral para la detección de flancos de subida (Trigger a Vpp / 2)
umbral = v_min + v_pp / 2;

flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);

t_flancos_ms = t_ms(flancos_idx);

% Período y Frecuencia
if length(t_flancos_ms) > 1
    periodo_ms = mean(diff(t_flancos_ms));
    frecuencia_hz = 1 / (periodo_ms * 1e-3);
else
    periodo_ms = NaN;
    frecuencia_hz = NaN;
end

```

```
%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES
```

```
figure('Name', 'Señal en el Tiempo - cuad2normal.csv', ...
```

```
    'Color', 'w', ...
```

```
    'Position', [100, 100, 1000, 500]);
```

```
hold on;
```

```
% Trazo principal de la señal
```

```
plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');
```

```
% Marcador Voltaje Máximo
```

```
plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...
```

```
    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));
```

```
% Marcador Voltaje Mínimo
```

```
plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...
```

```
    'DisplayName', sprintf('V_{mín} = %.2f V', v_min));
```

```
% Marcadores de flancos de subida
```

```
plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...
```

```
    'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');
```

```
%% 4. ANOTACIONES Y ETIQUETAS
```

```
info_str = {
```

```
    sprintf('V_{pp} = %.2f V', v_pp), ...
```

```
    sprintf('T = %.3f ms', periodo_ms), ...
```

```
    sprintf('f = %.1f kHz', frecuencia_hz / 1e3)
```

```
};
```

```
annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...
```

```
    'FitBoxToText', 'on', 'BackgroundColor', 'w', ...
```

```
    'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);
```

```
%% 5. ESTÉTICA Y FORMATO
```

```
grid on; grid minor; box on;
```

```
xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');
```

```
ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');
```

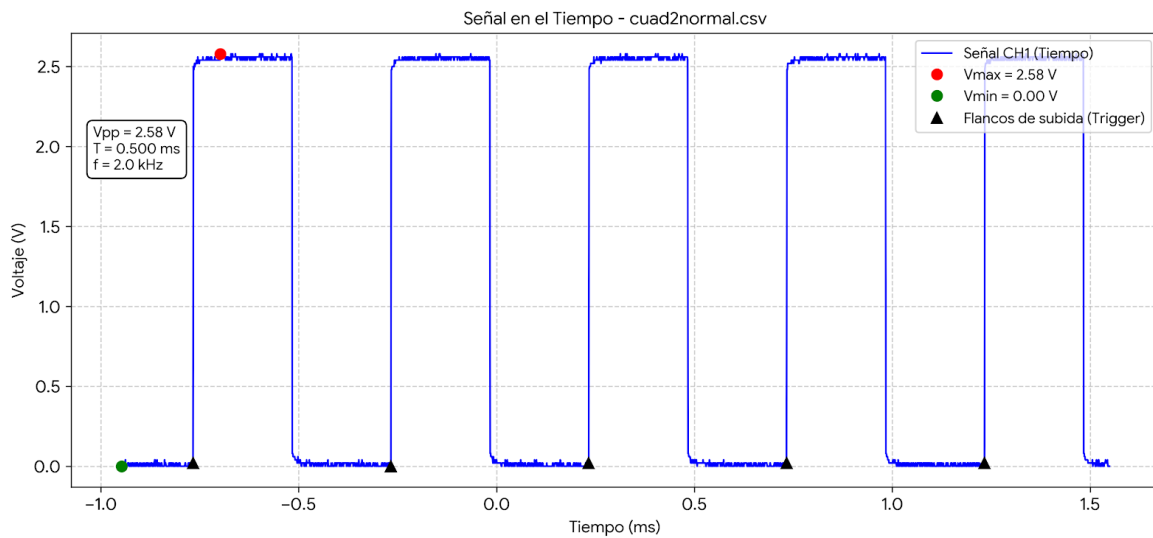
```
title('Señal en el Tiempo - cuad2normal.csv', 'FontSize', 12, 'FontWeight', 'bold');
```

```
legend('Location', 'northeast');
```

```
xlim([min(t_ms), max(t_ms)]);
```

```
ylim([v_min - 0.2, v_max + 0.3]);
```

```
hold off;
```



PULSO 1 FFT

```
%
```

```
% GRAFICACIÓN DE FFT CON MARCADORES (pulso1fft.csv)
```

```
%
```

```
clear; clc; close all;
```

```
%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV
```

```
filename = 'pulso1fft.csv';
```

```

if ~exist(filename, 'file')
    error('El archivo %s no se encuentra en el directorio actual.', filename);
end

fid = fopen(filename, 'r');
X = []; % Frecuencia (Hz)
Y = []; % Magnitud (dB)

while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
    if length(validos) >= 2
        X(end+1, 1) = validos(end-1);
        Y(end+1, 1) = validos(end);
    end
end

fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS PRINCIPALES)

% Se filtran picos con una prominencia mínima de 12 dB y magnitud superior a -30 dB
min_prominence = 12;
min_height = -30;

[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);

```

```

picos_X = X(locs);

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA

figure('Name', 'Espectro FFT - pulso1fft.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1050, 520]);

hold on;

% Trazo principal del espectro FFT
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');

% Marcadores sobre los picos principales
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...
    'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos)');

%% 4. ETIQUETADO DE LOS MARCADORES

for i = 1:length(picos_X)
    etiqueta = sprintf('%0.0f Hz\n%0.1f dB', picos_X(i), picos_Y(i));
    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...
        'FontSize', 7, ...
        'FontWeight', 'bold', ...
        'Color', 'k', ...
        'HorizontalAlignment', 'center', ...
        'VerticalAlignment', 'bottom');
end

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA

grid on; grid minor; box on;

xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');
title('Espectro de Frecuencia FFT - pulso1fft.csv', ...

```

```

'FontSize', 12, 'FontWeight', 'bold');

legend('Location', 'northeast')

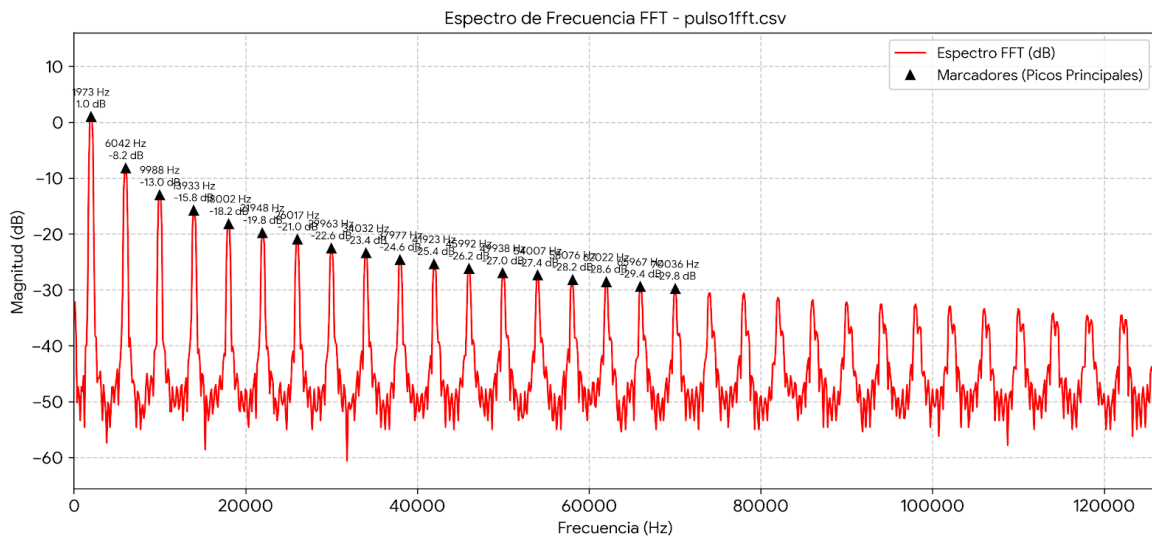
% Ajuste de límites de ejes

xlim([0, max(X)]);

ylim([min(Y) - 5, max(Y) + 15]);

hold off;

```



PULSO 1 NORMAL

```

%

% SEÑAL TEMPORAL CON MARCADORES (pulso1normal.csv)

%

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV

filename = 'pulso1normal.csv';

if ~exist(filename, 'file')

    error('El archivo %s no se encuentra en el directorio actual.', filename);

end

fid = fopen(filename, 'r');

```



```

t_raw = []; % Tiempo (s)
v_raw = []; % Voltaje (V)
while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));
    % Extrae los dos últimos valores numéricos de cada fila
    if length(validos) >= 2
        t_raw(end+1, 1) = validos(end-1);
        v_raw(end+1, 1) = validos(end);
    end
end
fclose(fid);
% Convertir tiempo a milisegundos (ms)
t_ms = t_raw * 1e3;
%% 2. CÁLCULO DE MÉTRICAS Y UBICACIÓN DE MARCADORES
[v_max, idx_max] = max(v_raw);
[v_min, idx_min] = min(v_raw);
v_pp = v_max - v_min;
% Umbral para detección de flancos de subida (Trigger a Vpp / 2)
umbral = v_min + v_pp / 2;
flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);
t_flancos_ms = t_ms(flancos_idx);
% Período y Frecuencia
if length(t_flancos_ms) > 1

```

```

    periodo_ms = mean(diff(t_flancos_ms));

    frecuencia_hz = 1 / (periodo_ms * 1e-3);

else

    periodo_ms = NaN;

    frecuencia_hz = NaN;

end

%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES

figure('Name', 'Señal en el Tiempo - pulso1normal.csv', ...

    'Color', 'w', ...

    'Position', [100, 100, 1000, 500]);

hold on;

% Trazo principal de la señal

plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');

% Marcador Voltaje Máximo

plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...

    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));

% Marcador Voltaje Mínimo

plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...

    'DisplayName', sprintf('V_{mín} = %.2f V', v_min));

% Marcadores de flancos de subida (Trigger)

plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...

    'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');

%% 4. ANOTACIONES Y ETIQUETAS

info_str = {

    sprintf('V_{pp} = %.2f V', v_pp), ...

    sprintf('T = %.3f ms', periodo_ms), ...

```

```

sprintf('f = %.1f kHz', frecuencia_hz / 1e3)

};

annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...

    'FitBoxToText', 'on', 'BackgroundColor', 'w', ...

    'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);

%% 5. ESTÉTICA Y FORMATO

grid on; grid minor; box on;

xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');

ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');

title('Señal en el Tiempo - pulso1normal.csv', 'FontSize', 12, 'FontWeight', 'bold');

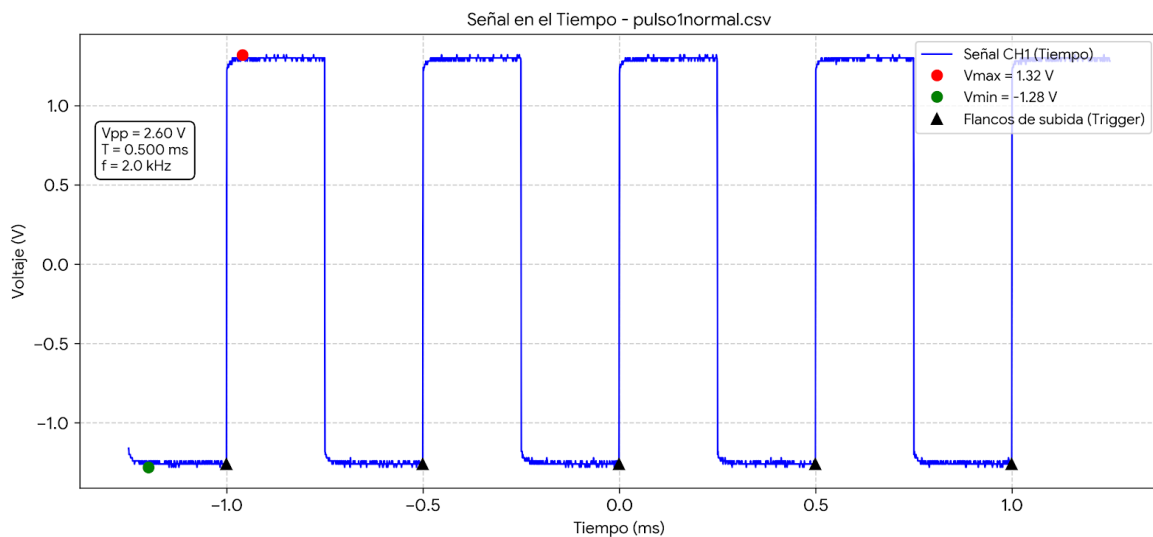
legend('Location', 'northeast');

xlim([min(t_ms), max(t_ms)]);

ylim([v_min - 0.2, v_max + 0.3]);

hold off;

```



PULSO 2 FFT

%

% GRAFICACIÓN DE FFT CON MARCADORES (pulso2fft.csv)

```

%

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV

filename = 'pulso2fft.csv';

if ~exist(filename, 'file')

    error('El archivo %s no se encuentra en el directorio actual.', filename);

end

fid = fopen(filename, 'r');

X = []; % Frecuencia (Hz)

Y = []; % Magnitud (dB)

while ~feof(fid)

    linea = fgetl(fid);

    partes = strsplit(linea, ',');

    nums = str2double(partes);

    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)

    if length(validos) >= 2

        X(end+1, 1) = validos(end-1);

        Y(end+1, 1) = validos(end);

    end

end

fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)

% Se filtran picos con una prominencia mínima de 15 dB y magnitud superior a -30 dB

min_prominence = 15;

min_height = -30

```

```

[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);
picos_X = X(locs);
%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA
figure('Name', 'Espectro FFT - pulso2fft.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1050, 520]);
hold on;
% Trazo principal del espectro FFT
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');
% Marcadores sobre los picos armónicos
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...
    'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');
%% 4. ETIQUETADO DE LOS MARCADORES
for i = 1:length(picos_X)
    etiqueta = sprintf('%0f Hz\n%.1f dB', picos_X(i), picos_Y(i));
    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...
        'FontSize', 7, ...
        'FontWeight', 'bold', ...
        'Color', 'k', ...
        'HorizontalAlignment', 'center', ...
        'VerticalAlignment', 'bottom');
end
%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA
grid on; grid minor; box on;

```

```
xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');
```

```
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');
```

```
title('Espectro de Frecuencia FFT - pulso2fft.csv', ...
```

```
    'FontSize', 12, 'FontWeight', 'bold');
```

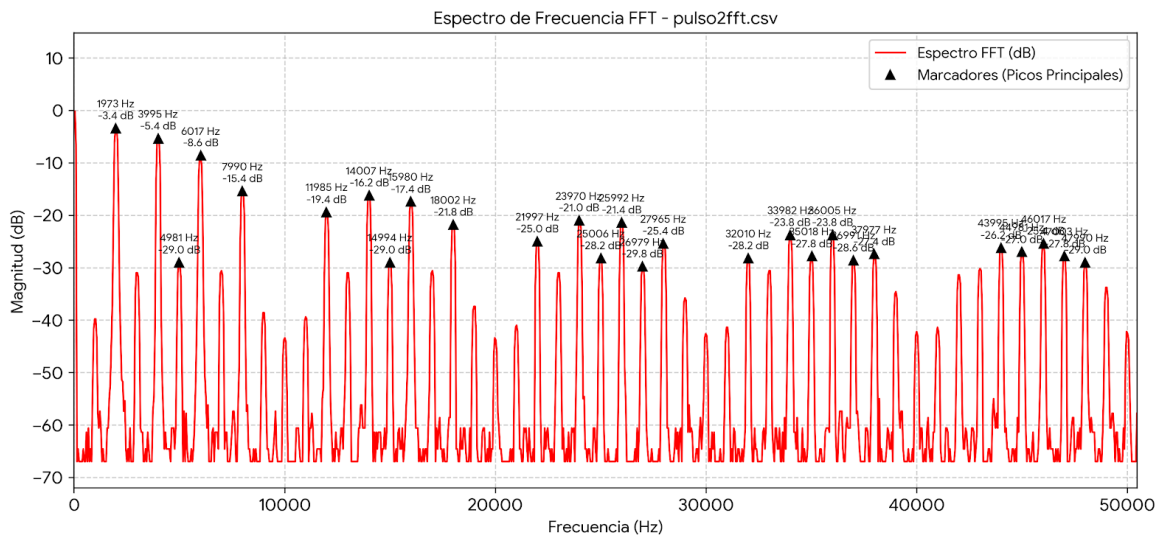
```
legend('Location', 'northeast');
```

```
% Ajuste de límites de ejes
```

```
xlim([0, max(X)]);
```

```
ylim([min(Y) - 5, max(Y) + 15]);
```

```
hold off;
```



PULSO 2 NORMAL

```
%
```

```
% GRAFICACIÓN DE FFT CON MARCADORES (pulso2normal.csv)
```

```
%
```

```
clear; clc; close all;
```

```
%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV
```

```
filename = 'pulso2normal.csv'
```

```
if ~exist(filename, 'file')
```

```

    error('El archivo %s no se encuentra en el directorio actual.', filename);
end
fid = fopen(filename, 'r');
X = []; % Frecuencia (Hz)
Y = []; % Magnitud (dB)
while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));
    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
    if length(validos) >= 2
        X(end+1, 1) = validos(end-1);
        Y(end+1, 1) = validos(end);
    end
end
fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS PRINCIPALES)
min_prominence = 12;
min_height = -30;
[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);
picos_X = X(locs);

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA
figure('Name', 'Espectro FFT - pulso2normal.csv', ...

```

```

        'Color', 'w', ...
        'Position', [100, 100, 1050, 520]);

hold on;

% Trazo principal del espectro FFT
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');

% Marcadores sobre los picos principales
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...
     'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Principales)');

%% 4. ETIQUETADO DE LOS MARCADORES
for i = 1:length(picos_X)
    etiqueta = sprintf('%0f Hz\n%.1f dB', picos_X(i), picos_Y(i));
    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...
         'FontSize', 7, ...
         'FontWeight', 'bold', ...
         'Color', 'k', ...
         'HorizontalAlignment', 'center', ...
         'VerticalAlignment', 'bottom');
end

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA
grid on; grid minor; box on;

xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');
title('Espectro de Frecuencia FFT - pulso2normal.csv', ...
      'FontSize', 12, 'FontWeight', 'bold');

legend('Location', 'northeast');

% Ajuste de límites de ejes

```



```
xlim([0, max(X)]);
```

```
ylim([min(Y) - 5, max(Y) + 15]);
```

```
hold off;
```

PULSO 3 FFT

```
%
```

```
% GRAFICACIÓN DE FFT CON MARCADORES (pulso3fft.csv)
```

```
%
```

```
clear; clc; close all;
```

```
%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV
```

```
filename = 'pulso3fft.csv';
```

```
if ~exist(filename, 'file')
```

```
    error('El archivo %s no se encuentra en el directorio actual.', filename);
```

```
end
```

```
fid = fopen(filename, 'r');
```

```
X = []; % Frecuencia (Hz)
```

```
Y = []; % Magnitud (dB)
```

```
while ~feof(fid)
```

```
    linea = fgetl(fid);
```

```
    partes = strsplit(linea, ',');
```

```
    nums = str2double(partes);
```

```
    validos = nums(~isnan(nums));
```

```
% Extraer los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
```

```
if length(validos) >= 2
```

```
    X(end+1, 1) = validos(end-1);
```

```
    Y(end+1, 1) = validos(end);
```

```
end
```

```

end

fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)

min_prominence = 15; % Prominencia mínima en dB

min_height = -30; % Altura mínima en dB

[picos_Y, locs] = findpeaks(Y, ...

    'MinPeakProminence', min_prominence, ...

    'MinPeakHeight', min_height);

picos_X = X(locs)

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA

figure('Name', 'Espectro FFT - pulso3fft.csv', ...

    'Color', 'w', ...

    'Position', [100, 100, 1050, 520]);

hold on;

% Trazo principal del espectro FFT

plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)')

% Marcadores sobre los picos armónicos

plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...

    'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');

%% 4. ETIQUETADO DE LOS MARCADORES

for i = 1:length(picos_X)

    etiqueta = sprintf('%.0f Hz\n%.1f dB', picos_X(i), picos_Y(i));

    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...

        'FontSize', 7, ...

        'FontWeight', 'bold', ...

        'Color', 'k', ...

```

```

'HorizontalAlignment', 'center', ...

'VerticalAlignment', 'bottom');

end

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA

grid on; grid minor; box on;

xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');

ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');

title('Espectro de Frecuencia FFT - pulso3fft.csv', ...

'FontSize', 12, 'FontWeight', 'bold');

legend('Location', 'northeast');

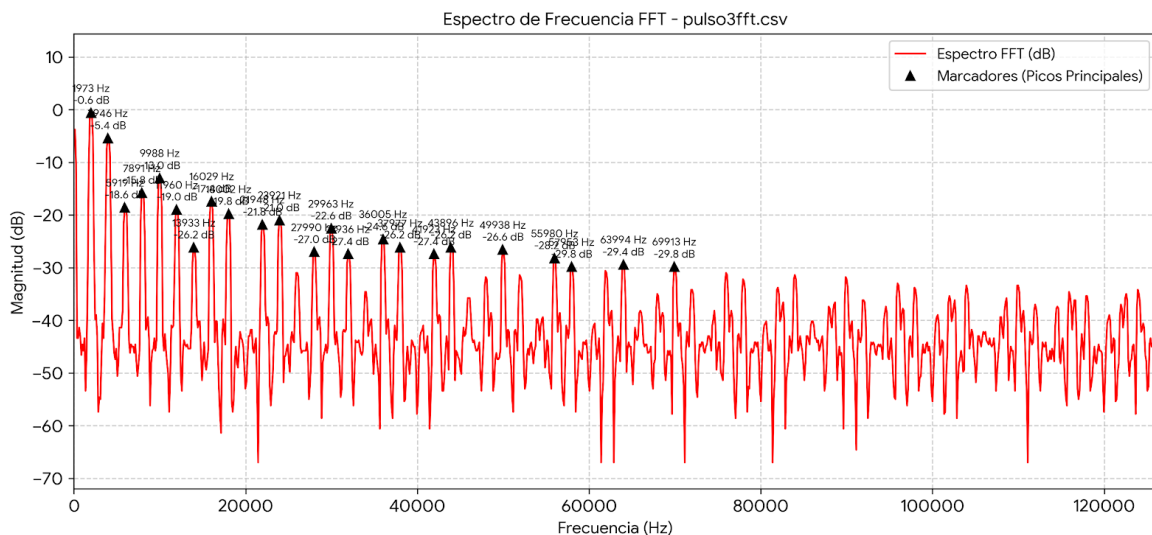
% Ajuste de límites de ejes

xlim([0, max(X)]);

ylim([min(Y) - 5, max(Y) + 15]);

hold off;

```



PULSO 3 NORMAL

%

% SEÑAL TEMPORAL CON MARCADORES (pulso3normal.csv)

```

%
clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV
filename = 'pulso3normal.csv';
if ~exist(filename, 'file')
    error('El archivo %s no se encuentra en el directorio actual.', filename);
end
fid = fopen(filename, 'r');
t_raw = []; % Tiempo (s)
v_raw = []; % Voltaje (V)
while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));
    % Extrae los dos últimos valores numéricos de cada fila (Tiempo y Voltaje)
    if length(validos) >= 2
        t_raw(end+1, 1) = validos(end-1);
        v_raw(end+1, 1) = validos(end);
    end
end
fclose(fid);
% Convertir tiempo a milisegundos (ms)
t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE MÉTRICAS Y UBICACIÓN DE MARCADORES
[v_max, idx_max] = max(v_raw);

```

```

[v_min, idx_min] = min(v_raw);

v_pp = v_max - v_min;

% Umbral para detección de flancos de subida (Trigger a Vpp / 2)
umbral = v_min + v_pp / 2;

flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);

t_flancos_ms = t_ms(flancos_idx);

% Período y Frecuencia
if length(t_flancos_ms) > 1
    periodo_ms = mean(diff(t_flancos_ms));
    frecuencia_hz = 1 / (periodo_ms * 1e-3);
else
    periodo_ms = NaN;
    frecuencia_hz = NaN;
end

%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES
figure('Name', 'Señal en el Tiempo - pulso3normal.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1000, 500]);

hold on;

% Trazo principal de la señal
plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');

% Marcador Voltaje Máximo
plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));

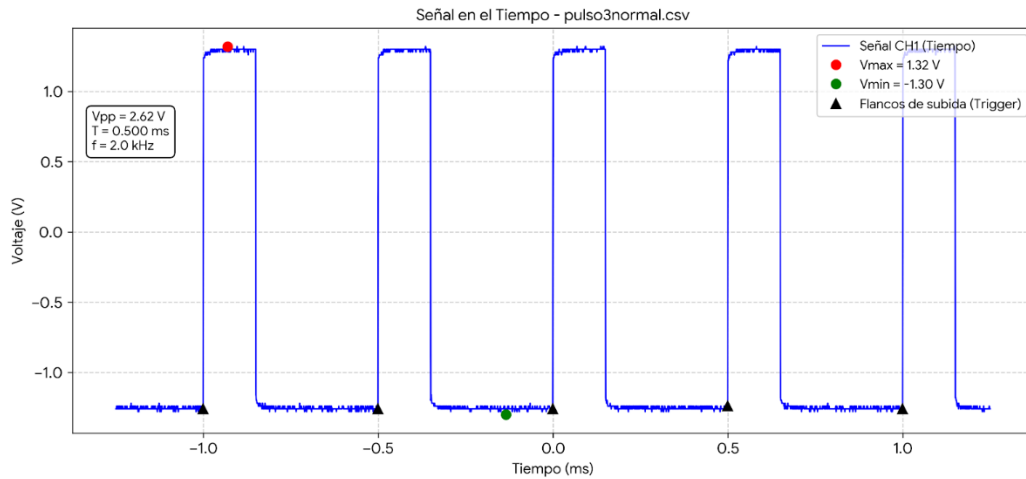
% Marcador Voltaje Mínimo
plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...

```

```

'DisplayName', sprintf('V_{mín} = %.2f V', v_min));
% Marcadores de flancos de subida (Trigger)
plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...
     'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');
%% 4. ANOTACIONES Y ETIQUETAS
info_str = {
    sprintf('V_{pp} = %.2f V', v_pp), ...
    sprintf('T = %.3f ms', periodo_ms), ...
    sprintf('f = %.1f kHz', frecuencia_hz / 1e3)
};
annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...
    'FitBoxToText', 'on', 'BackgroundColor', 'w', ...
    'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);
%% 5. ESTÉTICA Y FORMATO
grid on; grid minor; box on;
xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');
title('Señal en el Tiempo - pulso3normal.csv', 'FontSize', 12, 'FontWeight', 'bold');
legend('Location', 'northeast');
xlim([min(t_ms), max(t_ms)]);
ylim([v_min - 0.2, v_max + 0.3]);
hold off;

```



PULSO 4 FFT

%

% ESPECTRO FFT CON MARCADORES (pulso4fft.csv)

%

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV

filename = 'pulso4fft.csv';

if ~exist(filename, 'file')

error('El archivo %s no se encuentra en el directorio actual.', filename);

end

fid = fopen(filename, 'r');

X = []; % Frecuencia (Hz)

Y = []; % Magnitud (dB)

while ~feof(fid)

linea = fgetl(fid);

partes = strsplit(linea, ',');

nums = str2double(partes);

validos = nums(~isnan(nums));

```

% Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
if length(validos) >= 2
    X(end+1, 1) = validos(end-1);
    Y(end+1, 1) = validos(end);
end
end

fclose(fid)

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)
min_prominence = 15; % Prominencia mínima en dB
min_height = -30; % Altura mínima en dB
[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);
picos_X = X(locs);

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA
figure('Name', 'Espectro FFT - pulso4fft.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1050, 520]);
hold on;

% Trazo principal del espectro FFT
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');

% Marcadores sobre los picos armónicos
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...
    'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');

%% 4. ETIQUETADO DE LOS MARCADORES
for i = 1:length(picos_X)

```



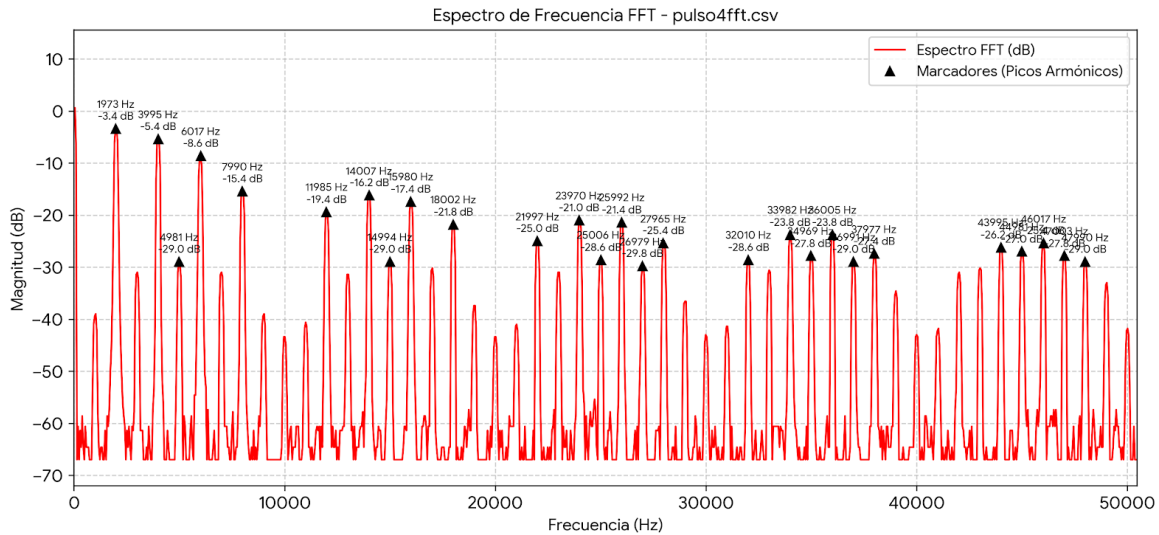
```

etiqueta = sprintf('%0f Hz\n%.1f dB', picos_X(i), picos_Y(i));
text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...
    'FontSize', 7, ...
    'FontWeight', 'bold', ...
    'Color', 'k', ...
    'HorizontalAlignment', 'center', ...
    'VerticalAlignment', 'bottom');
end

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA
grid on; grid minor; box on;
xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');
title('Espectro de Frecuencia FFT - pulso4fft.csv', ...
    'FontSize', 12, 'FontWeight', 'bold');
legend('Location', 'northeast');

% Ajuste de límites de ejes
xlim([0, max(X)]);
ylim([min(Y) - 5, max(Y) + 15]);
hold off;

```



PULSO 4 NORMAL

%

% SEÑAL EN EL TIEMPO CON MARCADORES (pulso4normal.csv)

%

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV

filename = 'pulso4normal.csv';

if ~exist(filename, 'file')

 error('El archivo %s no se encuentra en el directorio actual.', filename);

end

fid = fopen(filename, 'r');

t_raw = []; % Tiempo (s)

v_raw = []; % Voltaje (V)

while ~feof(fid)

 linea = fgetl(fid);

 partes = strsplit(linea, ',');

 nums = str2double(partes);

```

validos = nums(~isnan(nums));

% Extrae los dos últimos valores numéricos de cada fila (Tiempo y Voltaje)

if length(validos) >= 2

    t_raw(end+1, 1) = validos(end-1);

    v_raw(end+1, 1) = validos(end);

end

end

fclose(fid);

% Convertir tiempo a milisegundos (ms)

t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE MÉTRICAS Y MARCADORES

[v_max, idx_max] = max(v_raw);

[v_min, idx_min] = min(v_raw);

v_pp = v_max - v_min;

% Umbral para detección de flancos de subida (Trigger a Vpp / 2)

umbral = v_min + v_pp / 2;

flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);

t_flancos_ms = t_ms(flancos_idx);

% Período y Frecuencia

if length(t_flancos_ms) > 1

    periodo_ms = mean(diff(t_flancos_ms));

    frecuencia_hz = 1 / (periodo_ms * 1e-3);

else

    periodo_ms = NaN;

    frecuencia_hz = NaN;

end

```

%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES

```
figure('Name', 'Señal en el Tiempo - pulso4normal.csv', ...  
      'Color', 'w', ...  
      'Position', [100, 100, 1000, 500]);  
  
hold on;  
  
% Trazo principal de la señal  
plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');  
  
% Marcadores de Voltaje Máximo y Mínimo  
plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...  
     'DisplayName', sprintf('V_{máx} = %.2f V', v_max));  
plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...  
     'DisplayName', sprintf('V_{mín} = %.2f V', v_min));  
  
% Marcadores de flancos de subida  
plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...  
     'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');
```

%% 4. ANOTACIONES Y ETIQUETAS

```
info_str = {  
    sprintf('V_{pp} = %.2f V', v_pp), ...  
    sprintf('T = %.3f ms', periodo_ms), ...  
    sprintf('f = %.1f kHz', frecuencia_hz / 1e3)  
};  
  
annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...  
          'FitBoxToText', 'on', 'BackgroundColor', 'w', ...  
          'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);
```

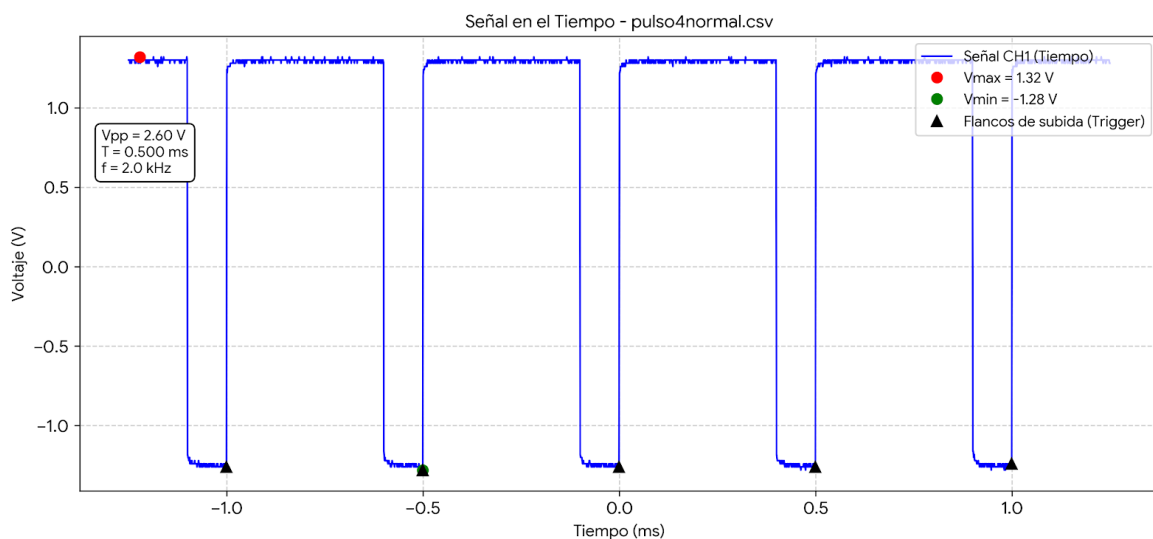
%% 5. ESTÉTICA Y FORMATO

```
grid on; grid minor; box on;
```

```

xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');
title('Señal en el Tiempo - pulso4normal.csv', 'FontSize', 12, 'FontWeight', 'bold');
legend('Location', 'northeast');
xlim([min(t_ms), max(t_ms)]);
ylim([v_min - 0.2, v_max + 0.3])
hold off;

```



SENO 1 FFT

```

%
% ESPECTRO FFT CON MARCADORES (Senoidal 1fft.csv)
%
clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV

filename = 'Senoidal 1fft.csv';

if ~exist(filename, 'file')
    error('El archivo %s no se encuentra en el directorio actual.', filename);
end

```

```

fid = fopen(filename, 'r');
X = []; % Frecuencia (Hz)
Y = []; % Magnitud (dB)
while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));
    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
    if length(validos) >= 2
        X(end+1, 1) = validos(end-1);
        Y(end+1, 1) = validos(end);
    end
end
fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)
min_prominence = 15; % Prominencia mínima en dB
min_height = -30; % Altura mínima en dB
[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);
picos_X = X(locs);

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA
figure('Name', 'Espectro FFT - Senoidal 1fft.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1050, 520]);

```

```

hold on;

% Trazo principal del espectro FFT
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');

% Marcadores sobre el pico fundamental
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...
     'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');

%% 4. ETIQUETADO DE LOS MARCADORES
for i = 1:length(picos_X)
    etiqueta = sprintf('%0f Hz\n%.1f dB', picos_X(i), picos_Y(i));
    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...
         'FontSize', 7, ...
         'FontWeight', 'bold', ...
         'Color', 'k', ...
         'HorizontalAlignment', 'center', ...
         'VerticalAlignment', 'bottom');
end

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA
grid on; grid minor; box on;

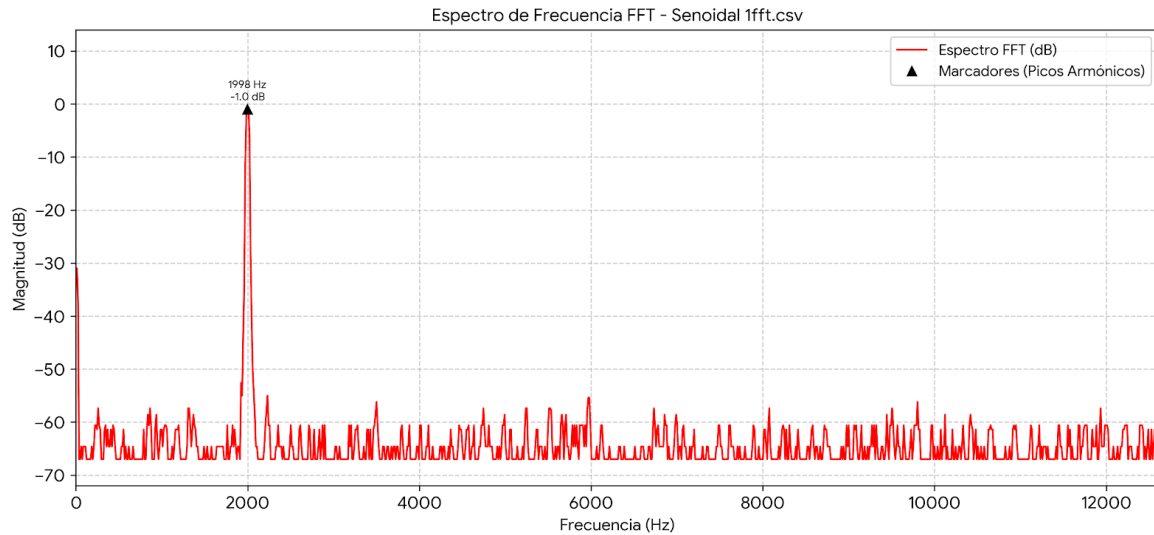
xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');
title('Espectro de Frecuencia FFT - Senoidal 1fft.csv', ...
      'FontSize', 12, 'FontWeight', 'bold');

legend('Location', 'northeast');

% Ajuste de límites de ejes
xlim([0, max(X)]);
ylim([min(Y) - 5, max(Y) + 15]);

```

hold off;



SENO 1 NORMAL

%

% SEÑAL EN EL TIEMPO CON MARCADORES (senoidal1normal.csv)

%

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV

filename = 'senoidal1normal.csv';

if ~exist(filename, 'file')

error('El archivo %s no se encuentra en el directorio actual.', filename);

end

fid = fopen(filename, 'r');

t_raw = []; % Tiempo (s)

v_raw = []; % Voltaje (V)

while ~feof(fid)

linea = fgetl(fid);

partes = strsplit(linea, ',');


```

    nums = str2double(partes);
    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Tiempo y Voltaje)
    if length(validos) >= 2
        t_raw(end+1, 1) = validos(end-1);
        v_raw(end+1, 1) = validos(end);
    end
end

fclose(fid);

% Convertir tiempo a milisegundos (ms)
t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE MÉTRICAS Y MARCADORES

[v_max, idx_max] = max(v_raw);
[v_min, idx_min] = min(v_raw);
v_pp = v_max - v_min;

% Umbral para detección de flancos de subida (Trigger a Vpp / 2)
umbral = v_min + v_pp / 2;

flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);

t_flancos_ms = t_ms(flancos_idx);

% Período y Frecuencia
if length(t_flancos_ms) > 1
    periodo_ms = mean(diff(t_flancos_ms));
    frecuencia_hz = 1 / (periodo_ms * 1e-3);
else
    periodo_ms = NaN;
    frecuencia_hz = NaN;
end

```

```

end

%%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES

figure('Name', 'Señal en el Tiempo - senoidal1normal.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1000, 500]);

hold on;

% Trazo principal de la señal senoidal
plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');

% Marcadores de Voltaje Máximo y Mínimo
plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));
plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{mín} = %.2f V', v_min));

% Marcadores de flancos de subida (Trigger)
plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...
    'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');

%%% 4. ANOTACIONES Y ETIQUETAS

info_str = {
    sprintf('V_{pp} = %.2f V', v_pp), ...
    sprintf('T = %.3f ms', periodo_ms), ...
    sprintf('f = %.1f kHz', frecuencia_hz / 1e3)
};

annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...
    'FitBoxToText', 'on', 'BackgroundColor', 'w', ...
    'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);

```

```
%% 5. ESTÉTICA Y FORMATO
```

```
grid on; grid minor; box on;
```

```
xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');
```

```
ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');
```

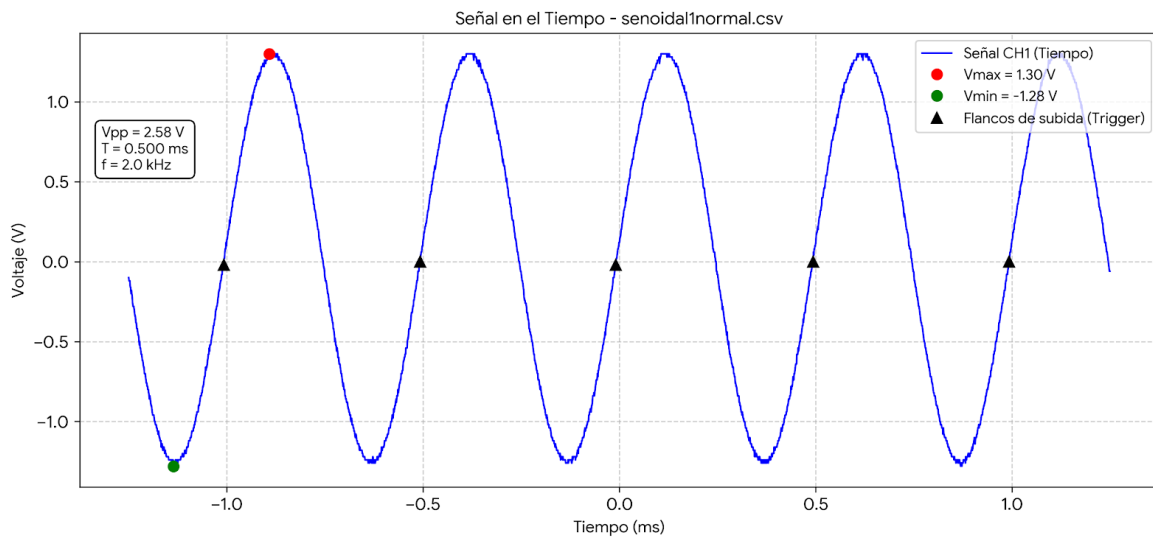
```
title('Señal en el Tiempo - senoidal1normal.csv', 'FontSize', 12, 'FontWeight', 'bold');
```

```
legend('Location', 'northeast');
```

```
xlim([min(t_ms), max(t_ms)]);
```

```
ylim([v_min - 0.2, v_max + 0.3]);
```

```
hold off;
```



SENO 2 FFT

```
%
```

```
% ESPECTRO FFT CON MARCADORES (sen2fft.csv)
```

```
%
```

```
clear; clc; close all;
```

```
%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV
```

```
filename = 'sen2fft.csv';
```

```

if ~exist(filename, 'file')
    error('El archivo %s no se encuentra en el directorio actual.', filename);
end

fid = fopen(filename, 'r');
X = []; % Frecuencia (Hz)
Y = []; % Magnitud (dB)

while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
    if length(validos) >= 2
        X(end+1, 1) = validos(end-1);
        Y(end+1, 1) = validos(end);
    end
end

fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)

min_prominence = 15; % Prominencia mínima en dB
min_height = -30; % Altura mínima en dB

[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);

picos_X = X(locs);

```

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA

```
figure('Name', 'Espectro FFT - sen2fft.csv', ...  
      'Color', 'w', ...  
      'Position', [100, 100, 1050, 520]);  
  
hold on;  
  
% Trazo principal del espectro FFT  
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');  
  
% Marcadores sobre los picos armónicos/fundamentales  
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...  
     'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');
```

%% 4. ETIQUETADO DE LOS MARCADORES

```
for i = 1:length(picos_X)  
    etiqueta = sprintf('%.0f Hz\n%.1f dB', picos_X(i), picos_Y(i));  
    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...  
        'FontSize', 7.5, ...  
        'FontWeight', 'bold', ...  
        'Color', 'k', ...  
        'HorizontalAlignment', 'center', ...  
        'VerticalAlignment', 'bottom');
```

en

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA

```
grid on; grid minor; box on;  
  
xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');  
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');  
  
title('Espectro de Frecuencia FFT - sen2fft.csv', ...  
     'FontSize', 12, 'FontWeight', 'bold');
```

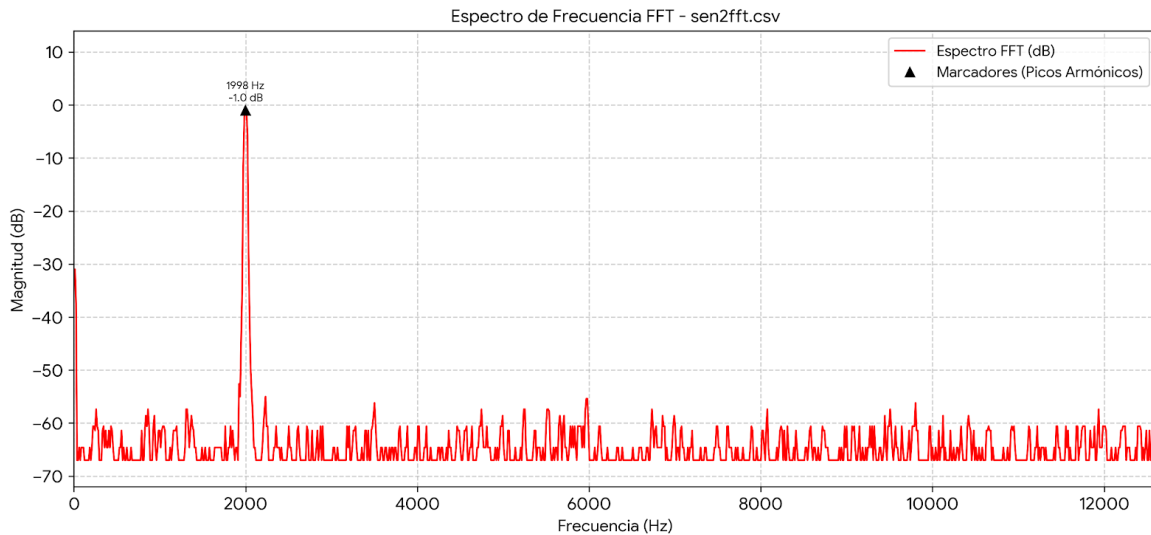
```
legend('Location', 'northeast');
```

```
% Ajuste de límites de ejes
```

```
xlim([0, max(X)]);
```

```
ylim([min(Y) - 5, max(Y) + 15]);
```

```
hold off;
```



SENO 2 NORMAL

```
%
```

```
% SEÑAL EN EL TIEMPO CON MARCADORES (senoidal2normal.csv)
```

```
%
```

```
clear; clc; close all;
```

```
%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV
```

```
filename = 'senoidal2normal.csv';
```

```
if ~exist(filename, 'file')
```

```
    error('El archivo %s no se encuentra en el directorio actual.', filename);
```

```
end
```

```
fid = fopen(filename, 'r');
```

```
t_raw = []; % Tiempo (s)
```

```

v_raw = []; % Voltaje (V)

while ~feof(fid)

    linea = fgetl(fid);

    partes = strsplit(linea, ',');

    nums = str2double(partes);

    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Tiempo y Voltaje)

    if length(validos) >= 2

        t_raw(end+1, 1) = validos(end-1);

        v_raw(end+1, 1) = validos(end);

    end

end

fclose(fid);

% Convertir tiempo a milisegundos (ms)

t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE MÉTRICAS Y MARCADORES

[v_max, idx_max] = max(v_raw);

[v_min, idx_min] = min(v_raw);

v_pp = v_max - v_min;

% Umbral para detección de flancos de subida (Trigger a Vpp / 2)

umbral = v_min + v_pp / 2;

flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);

t_flancos_ms = t_ms(flancos_idx);

% Período y Frecuencia

if length(t_flancos_ms) > 1

    periodo_ms = mean(diff(t_flancos_ms));

```

```

    frecuencia_hz = 1 / (periodo_ms * 1e-3);
else
    periodo_ms = NaN;
    frecuencia_hz = NaN;
end

%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES

figure('Name', 'Señal en el Tiempo - senoidal2normal.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1000, 500]);

hold on;

% Trazo principal de la señal senoidal
plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');

% Marcadores de Voltaje Máximo y Mínimo
plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));
plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{mín} = %.2f V', v_min));

% Marcadores de flancos de subida (Trigger)
plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...
    'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');

%% 4. ANOTACIONES Y ETIQUETAS

info_str = {
    sprintf('V_{pp} = %.2f V', v_pp), ...
    sprintf('T = %.3f ms', periodo_ms), ...
    sprintf('f = %.1f kHz', frecuencia_hz / 1e3)
};

```



```

annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...
    'FitBoxToText', 'on', 'BackgroundColor', 'w', ...
    'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);

%% 5. ESTÉTICA Y FORMATO

grid on; grid minor; box on;

xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');

ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');

title('Señal en el Tiempo - senoidal2normal.csv', 'FontSize', 12, 'FontWeight', 'bold');

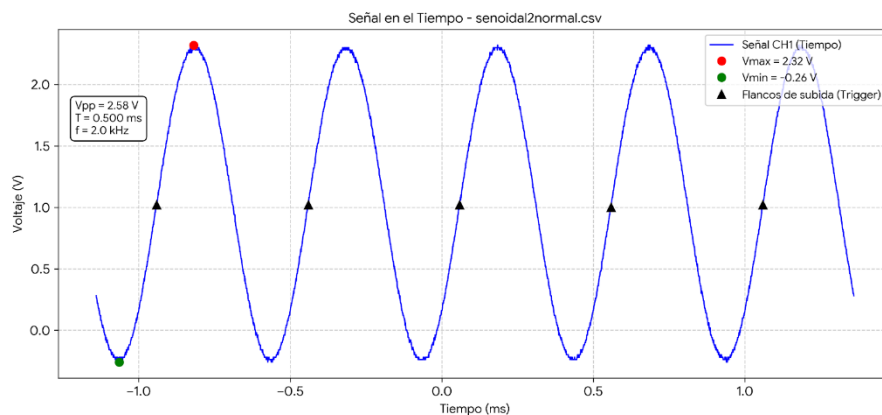
legend('Location', 'northeast');

xlim([min(t_ms), max(t_ms)]);

ylim([v_min - 0.2, v_max + 0.3]);

hold off;

```



TRIANGULO 1 FFT

```

%

% ESPECTRO FFT CON MARCADORES (triang1fft.csv)

%

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV

filename = 'triang1fft.csv';

```

```

if ~exist(filename, 'file')
    error('El archivo %s no se encuentra en el directorio actual.', filename);
end

fid = fopen(filename, 'r');
X = []; % Frecuencia (Hz)
Y = []; % Magnitud (dB)

while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
    if length(validos) >= 2
        X(end+1, 1) = validos(end-1);
        Y(end+1, 1) = validos(end);
    end
end

fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)

min_prominence = 12; % Prominencia mínima en dB
min_height = -30; % Altura mínima en dB

[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...
    'MinPeakHeight', min_height);

picos_X = X(locs);

```

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA

```
figure('Name', 'Espectro FFT - triang1fft.csv', ...  
      'Color', 'w', ...  
      'Position', [100, 100, 1050, 520]);  
  
hold on;  
  
% Trazo principal del espectro FFT  
plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');  
  
% Marcadores sobre los picos armónicos  
plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...  
     'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');
```

%% 4. ETIQUETADO DE LOS MARCADORES

```
for i = 1:length(picos_X)  
    etiqueta = sprintf('%.0f Hz\n%.1f dB', picos_X(i), picos_Y(i));  
    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...  
        'FontSize', 7.5, ...  
        'FontWeight', 'bold', ...  
        'Color', 'k', ...  
        'HorizontalAlignment', 'center', ...  
        'VerticalAlignment', 'bottom');  
end
```

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA

```
grid on; grid minor; box on;  
  
xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');  
ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');  
  
title('Espectro de Frecuencia FFT - triang1fft.csv', ...  
      'FontSize', 12, 'FontWeight', 'bold');
```

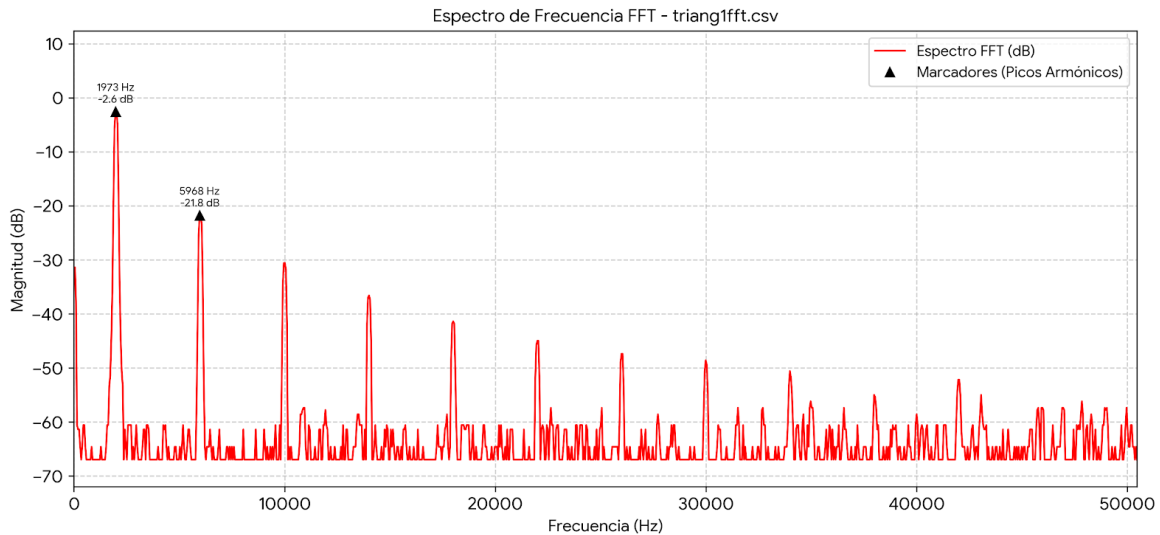
```
legend('Location', 'northeast');
```

```
% Ajuste de límites de ejes
```

```
xlim([0, max(X)]);
```

```
ylim([min(Y) - 5, max(Y) + 15]);
```

```
hold off;
```



TRIANGULO 1 NORMAL

```
%
```

```
% SEÑAL EN EL TIEMPO CON MARCADORES (triang1normal.csv)
```

```
%
```

```
clear; clc; close all;
```

```
%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV
```

```
filename = 'triang1normal.csv';
```

```
if ~exist(filename, 'file')
```

```
    error('El archivo %s no se encuentra en el directorio actual.', filename);
```

```
end
```

```
fid = fopen(filename, 'r');
```

```
t_raw = []; % Tiempo (s)
```

```

v_raw = []; % Voltaje (V)

while ~feof(fid)

    linea = fgetl(fid);

    partes = strsplit(linea, ',');

    nums = str2double(partes);

    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Tiempo y Voltaje)

    if length(validos) >= 2

        t_raw(end+1, 1) = validos(end-1);

        v_raw(end+1, 1) = validos(end);

    end

end

fclose(fid);

% Convertir tiempo a milisegundos (ms)

t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE MÉTRICAS Y MARCADORES

[v_max, idx_max] = max(v_raw);

[v_min, idx_min] = min(v_raw);

v_pp = v_max - v_min;

% Umbral para detección de flancos de subida (Trigger a Vpp / 2)

umbral = v_min + v_pp / 2;

flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);

t_flancos_ms = t_ms(flancos_idx);

% Período y Frecuencia

if length(t_flancos_ms) > 1

    periodo_ms = mean(diff(t_flancos_ms));

```

```

    frecuencia_hz = 1 / (periodo_ms * 1e-3);
else
    periodo_ms = NaN;
    frecuencia_hz = NaN;
end

%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES
figure('Name', 'Señal en el Tiempo - triang1normal.csv', ...
    'Color', 'w', ...
    'Position', [100, 100, 1000, 500]);
hold on;

% Trazo principal de la señal triangular
plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');

% Marcadores de Voltaje Máximo y Mínimo
plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));
plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...
    'DisplayName', sprintf('V_{mín} = %.2f V', v_min));

% Marcadores de flancos de subida
plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...
    'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');

%% 4. ANOTACIONES Y ETIQUETAS
info_str = {
    sprintf('V_{pp} = %.2f V', v_pp), ...
    sprintf('T = %.3f ms', periodo_ms), ...
    sprintf('f = %.1f kHz', frecuencia_hz / 1e3)
};

```

```

annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...
    'FitBoxToText', 'on', 'BackgroundColor', 'w', ...
    'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);

%% 5. ESTÉTICA Y FORMATO

grid on; grid minor; box on;

xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');

ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');

title('Señal en el Tiempo - triang1normal.csv', 'FontSize', 12, 'FontWeight', 'bold');

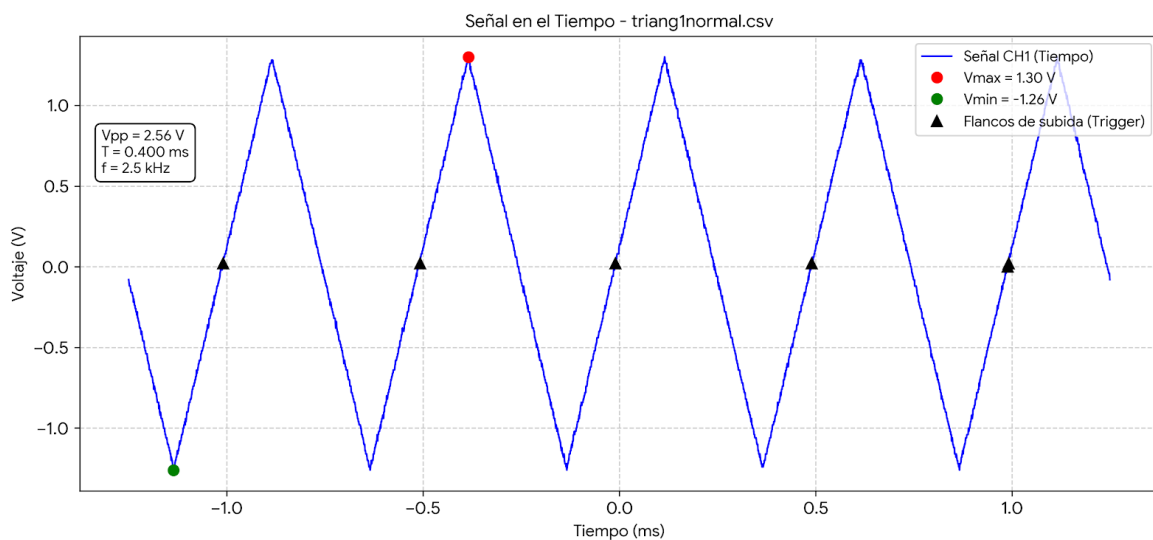
legend('Location', 'northeast');

xlim([min(t_ms), max(t_ms)]);

ylim([v_min - 0.2, v_max + 0.3]);

hold off;

```



TRIANGULO 2 FFT

```

%

% ESPECTRO FFT CON MARCADORES (triang2fft.csv)

%

```

```

clear; clc; close all;

%% 1. LECTURA Y PROCESAMIENTO ROBUSTO DEL ARCHIVO CSV

filename = 'triang2fft.csv';

if ~exist(filename, 'file')
    error('El archivo %s no se encuentra en el directorio actual.', filename);
end

fid = fopen(filename, 'r');

X = []; % Frecuencia (Hz)
Y = []; % Magnitud (dB)

while ~feof(fid)
    linea = fgetl(fid);
    partes = strsplit(linea, ',');
    nums = str2double(partes);
    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Frecuencia y Magnitud)
    if length(validos) >= 2
        X(end+1, 1) = validos(end-1);
        Y(end+1, 1) = validos(end);
    end
end

fclose(fid);

%% 2. DETECCIÓN AUTOMÁTICA DE MARCADORES (PICOS ARMÓNICOS)

min_prominence = 12; % Prominencia mínima en dB

min_height = -30; % Altura mínima en dB

[picos_Y, locs] = findpeaks(Y, ...
    'MinPeakProminence', min_prominence, ...

```



```

    'MinPeakHeight', min_height);

picos_X = X(locs);

%% 3. CREACIÓN Y CONFIGURACIÓN DE LA FIGURA

figure('Name', 'Espectro FFT - triang2fft.csv', ...

    'Color', 'w', ...

    'Position', [100, 100, 1050, 520]);

hold on;

% Trazo principal del espectro FFT

plot(X, Y, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Espectro FFT (dB)');

% Marcadores sobre los picos armónicos

plot(picos_X, picos_Y, 'k^', 'MarkerFaceColor', 'k', ...

    'MarkerSize', 6, 'DisplayName', 'Marcadores (Picos Armónicos)');

%% 4. ETIQUETADO DE LOS MARCADORES

for i = 1:length(picos_X)

    etiqueta = sprintf('%0f Hz\n%.1f dB', picos_X(i), picos_Y(i));

    text(picos_X(i), picos_Y(i) + 1.5, etiqueta, ...

        'FontSize', 7.5, ...

        'FontWeight', 'bold', ...

        'Color', 'k', ...

        'HorizontalAlignment', 'center', ...

        'VerticalAlignment', 'bottom');

end

%% 5. FORMATO Y ESTÉTICA DE LA GRÁFICA

grid on; grid minor; box on;

xlabel('Frecuencia (Hz)', 'FontSize', 11, 'FontWeight', 'bold');

ylabel('Magnitud (dB)', 'FontSize', 11, 'FontWeight', 'bold');

```

```
title('Espectro de Frecuencia FFT - triang2fft.csv', ...
```

```
    'FontSize', 12, 'FontWeight', 'bold');
```

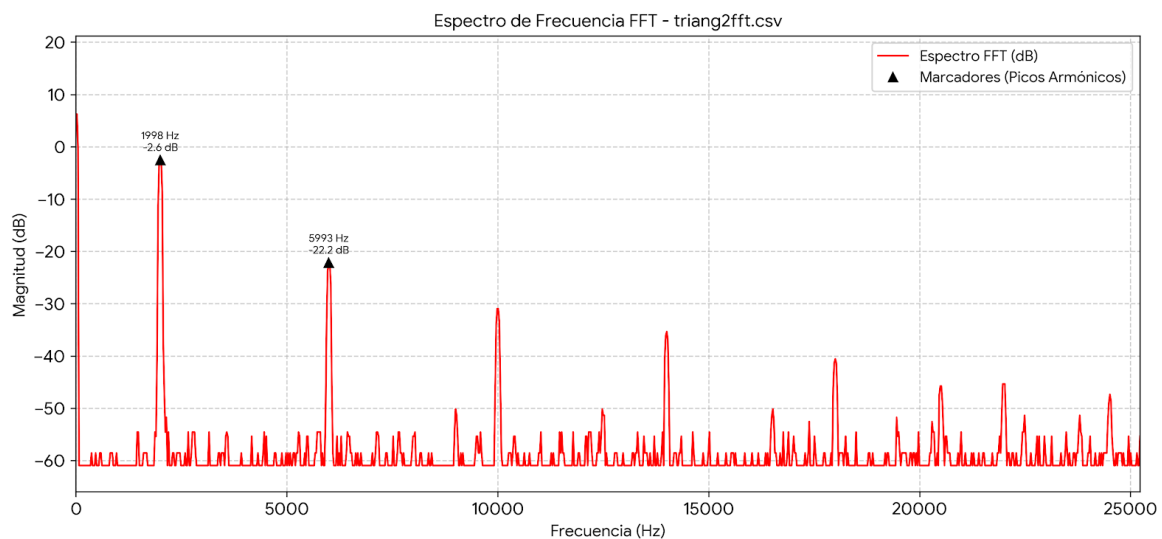
```
legend('Location', 'northeast');
```

```
% Ajuste de límites de ejes
```

```
xlim([0, max(X)]);
```

```
ylim([min(Y) - 5, max(Y) + 15]);
```

```
hold off;
```



TRIANGULO 2 NORMAL

```
%
```

```
% SEÑAL EN EL TIEMPO CON MARCADORES (triang2normal.csv)
```

```
%
```

```
clear; clc; close all;
```

```
%% 1. LECTURA Y PROCESAMIENTO DEL ARCHIVO CSV
```

```
filename = 'triang2normal.csv';
```

```
if ~exist(filename, 'file')
```

```
    error('El archivo %s no se encuentra en el directorio actual.', filename);
```

```
end
```

```

fid = fopen(filename, 'r');

t_raw = []; % Tiempo (s)
v_raw = []; % Voltaje (V)

while ~feof(fid)

    linea = fgetl(fid);

    partes = strsplit(linea, ',');

    nums = str2double(partes);

    validos = nums(~isnan(nums));

    % Extrae los dos últimos valores numéricos de cada fila (Tiempo y Voltaje)

    if length(validos) >= 2

        t_raw(end+1, 1) = validos(end-1);

        v_raw(end+1, 1) = validos(end);

    end

end

fclose(fid)

% Convertir tiempo a milisegundos (ms)

t_ms = t_raw * 1e3;

%% 2. CÁLCULO DE MÉTRICAS Y MARCADORES

[v_max, idx_max] = max(v_raw);

[v_min, idx_min] = min(v_raw);

v_pp = v_max - v_min

% Umbral para detección de flancos de subida (Trigger a Vpp / 2)

umbral = v_min + v_pp / 2;

flancos_idx = find(v_raw(1:end-1) < umbral & v_raw(2:end) >= umbral);

t_flancos_ms = t_ms(flancos_idx);

```

```

% Período y Frecuencia

if length(t_flancos_ms) > 1

    periodo_ms = mean(diff(t_flancos_ms));

    frecuencia_hz = 1 / (periodo_ms * 1e-3);

else

    periodo_ms = NaN;

    frecuencia_hz = NaN;

end

%% 3. CREACIÓN DE LA GRÁFICA Y MARCADORES

figure('Name', 'Señal en el Tiempo - triang2normal.csv', ...

    'Color', 'w', ...

    'Position', [100, 100, 1000, 500]);

hold on

% Trazo principal de la señal triangular

plot(t_ms, v_raw, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Señal CH1 (Tiempo)');

% Marcadores de Voltaje Máximo y Mínimo

plot(t_ms(idx_max), v_max, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7, ...

    'DisplayName', sprintf('V_{máx} = %.2f V', v_max));

plot(t_ms(idx_min), v_min, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 7, ...

    'DisplayName', sprintf('V_{mín} = %.2f V', v_min));

% Marcadores de flancos de subida

plot(t_flancos_ms, v_raw(flancos_idx), 'k^', 'MarkerFaceColor', 'k', ...

    'MarkerSize', 6, 'DisplayName', 'Flancos de subida (Trigger)');

%% 4. ANOTACIONES Y ETIQUETAS

info_str = {

    sprintf('V_{pp} = %.2f V', v_pp), ...

```

```

sprintf('T = %.3f ms', periodo_ms), ...

sprintf('f = %.1f kHz', frecuencia_hz / 1e3)

};

annotation('textbox', [0.14, 0.68, 0.22, 0.18], 'String', info_str, ...

    'FitBoxToText', 'on', 'BackgroundColor', 'w', ...

    'EdgeColor', 'k', 'FontWeight', 'bold', 'FontSize', 9);

%% 5. ESTÉTICA Y FORMATO

grid on; grid minor; box on;

xlabel('Tiempo (ms)', 'FontSize', 11, 'FontWeight', 'bold');

ylabel('Voltaje (V)', 'FontSize', 11, 'FontWeight', 'bold');

title('Señal en el Tiempo - triang2normal.csv', 'FontSize', 12, 'FontWeight', 'bold')

legend('Location', 'northeast');

xlim([min(t_ms), max(t_ms)]);

ylim([v_min - 0.2, v_max + 0.3]);

hold off;

```

